Arabian Journal for Science and Engineering
https://doi.org/10.1007/s13369-021-05427-8

RESEARCH ARTICLE-COMPUTER ENGINEERING AND COMPUTER SCIENCE

An Improved Blockchain Consensus Algorithm Based on Raft

Wei Fu[1] · Xuefeng Wei[1] · Shihua Tong[2]

Received: 20 October 2020 / Accepted: 28 January 2021
© King Fahd University of Petroleum & Minerals 2021

Abstract
To address the problem of performance degradation caused by the blockchain backup mechanism as well as the high throughput and low latency requirements characteristic of blockchain applications, this paper optimizes the Raft consensus algorithm for the Hyperledger Fabric platform in terms of both log replication and leader election. In the log replication phase, the improvement of log replication based on apportionment idea aims to reduce the communication complexity of leader node by involving peer node in distributing log information. In the leader election phase, the improvement of leader election based on vote change mechanism changes the vote affiliation of peer node based on a comparison of votes from candidate nodes, aims to reach consensus in a round of election and reduces the election time. Through the performance test of the blockchain, it is demonstrated that the improved AdRaft algorithm improves 5.8% in throughput and reduces 1.3% in latency over the original Raft algorithm.

Keywords Raft · Log replication · Leader election · Vote change mechanism · Vote affiliation

1 Introduction

A blockchain can be referred to as a distributed ledger, where the data and transactions are not under the control of any third party. As early as 1982, Lamport described the Byzantine problem as the problem of "how to achieve trustworthy communication in an adversarial network environment" [1]. In later research, Castro and Liskov [2] proposed the practical "Practical Byzantine Fault Tolerance Algorithm" (PBFT), which reduced the complexity of Lamport's paper from exponential to polynomial level. It was not until 2008 that Satoshi Nakamoto proposed the Bitcoin system using the PoW algorithm as the bookkeeping mechanism and Merkel trees as the data structure to record data efficiently through timestamps [3], it marked the birth of the first generation of blockchain. The second-generation blockchain is a combination of the smart contract concept proposed by Nick Szabo [4] and the P2P, POW consensus algorithm, and Merkel tree technologies used in Bitcoin to build a decentralized trusted execution system without a third-party regulatory platform. Since 2016, more and more blockchain applications will fall into place as major companies propose and refine new architectures and consensus algorithms for their respective blockchain technologies 3.0.

The consensus algorithm is a solution to the problem of message consistency in a blockchain network with respect to how individual nodes reach message consistency. Consensus algorithm is the most important factor of the entire blockchain system, for the reason that its efficiency determines the blockchain's performance directly.

In this paper, we adopt the Hyperledger Fabric architecture as the main framework of the blockchain and improve on the platform's Raft algorithm. Raft-based improved consensus algorithm adopts the idea of equal sharing to solve the problem of performance bottlenecks for a single leader node, and also adopts an improved vote change mechanism to solve the problem of multiple rounds in the leader election phase.

The rest of this paper is organized as follows: Sect. 2 describes current status of research on consensus algorithms as well as some deficiencies in the log replication phase and the leader election phase. Section 3 introduces the

This work was supported by Chongqing technology innovation and application development special key projects under award cstc2018jszx-cyzdX0036.

✉ Xuefeng Wei
1198136191@qq.com

1 Chongqing University of Posts and Telecommunications, Chongqing, China

2 Chongqing College of Electronic Engineering, Chongqing, China

Published online: 16 February 2021

optimization idea and the process of the improved AdRaft algorithm, then describes it by pseudocode. Section 4 goes on to demonstrate the consistency of the improved algorithm by describing various restriction mechanisms to complement and refine the rules of the AdRaft algorithm. Section 5 uses Caliper, Hyperledger's official testing tool, to test and validate the log replication and leader election, respectively. Section 5 concludes the paper.

2 Related Work

2.1 Current Status of Research on Consensus Algorithms

In the public domain, most of the existing mainstream consensus algorithms are combined with fragmentation techniques to improve network throughput and reduce latency. Zilliqa blockchain [5] adopts a hybrid algorithm with PoW algorithm as the admission mechanism and PBFT algorithm as the block consensus, and then uses node random fragmentation mechanism to ensure network security. Algorand algorithm [6] proposes the use of verifiable random function (VRF) to elect the consensus committee, and use the committee to conduct multiple rounds of Byzantine consensus. Conflux [7], which is a fast, scalable and decentralized blockchain system that optimistically process concurrent blocks without discarding any as forks. The Conflux consensus protocol represents relationships between blocks as a direct acyclic graph and achieves consensus on a total order of the blocks.

Dr. Jiaping Wang proposed a Monoxide expansion scheme [8], based on the PoW consensus algorithm as a blueprint, independently dividing the three levels of block verification, storage and ledger state update, and using a "crossbow mining" mechanism to ensure the fragmentation consensus group computing power dispersion, preventing 51% of the computing power attacks, using fragmentation technology Breakthrough FLP Impossible Consensus.

In the federation chain domain, the HQ [9] and Zyzzyva [10] protocols simplify the PBFT algorithm consensus step to improve performance by assuming an ideal network environment. The FastBFT algorithm [11] uses message aggregation to improve the communication complexity of PBFT algorithms based on a hardware trusted execution environment. The Swirlds algorithm [12] innovatively uses a directed acyclic graph (DAG) network structure and uses gossip as a communication protocol to achieve asynchronous consensus of leaderless nodes. The HoneyBadger BFT algorithm [13], on the other hand, relies on the asynchronous consensus transaction group and apportioned transaction mechanism to ensure the BFT trusted operation mechanism through random selection and threshold encryption algorithm.

The field of consensus algorithms for non-Byzantine models has undergone a series of improvements such as Paxos [14], ZAB [15], and Chubby [16]. D.Ongaro and J.Ousterhout proposed the Raft algorithm in 2014 to address problems such as the difficulty of understanding and practicing Paxos-like algorithms [17]. Currently, there is a relative lack of research on improving Raft's algorithm at home and abroad. In the field of log compression and encoding storage, Wang Zizhong et al. [18] proposed to improve the Raft algorithm by adopting fault-tolerance of log messages through deletion correction codes. In terms of Byzantine fault tolerance, Tan and Hu [19] introduced digital signatures and nested hash mechanisms to make Raft byzantine fault tolerant. Wang Rihong, Zhang Lifeng [20] combined Raft with a BLS signature mechanism to resist malicious nodes. In terms of performance improvement, Wang Rihong, Zhang Lifeng [21] used the Kademlia algorithm to network Raft's underlying network and to reduce the communication complexity. Kunpeng Dai [22] proposed an OLR-Raft improvement algorithm to optimize the algorithm through log addressing optimization, cache logging and parallel log replication. The log addressing optimization can only solve the peer node non-conflicting log backtracking efficiency; the cache logging scheme can increase the throughput of a round of consensus, but does not really solve the leader node communication bottleneck problem; the parallel log replication is not optimally effective due to the fact that the RPC call time is much higher than the log landing time. Wang [23] introduced a new conflictIndex array to represent the log conflict state based on Raft algorithm to optimize the log backtracking problem. Its overall algorithm does not make major improvements in throughput to improve consensus performance.

2.2 Problems with the Raft Algorithm

2.2.1 Problems with the Log Replication Phase

Whether it is PBFT, Zookeeper or Raft algorithm, the core idea is a class of distributed consensus algorithms based on strong leader nodes. Raft algorithm strengthens the leader position and splits the overall algorithm into two parts: log replication and leader election.

Synchronize logs from the leader node to the follower node when the leader is functioning normally. When the leader crashes, the election algorithm picks a new leader node.

From the realizability point of view of the algorithm, a single leader node ensures the determinism of the distributed network and prevents the consensus bifurcation of the network. When the request from the client is received, the

leader node adds the request to the log, and then launches the log replication request to other follower nodes in parallel. When this log is replicated to more than half of the nodes, the leader node then returns the log application state result to the client. If the follower node is down, slow, or loses packets, the leader node will keep retrying until all follower nodes eventually replicate all log entries. The log replication process is shown in Fig. 1.

But with the increase in the number of follower nodes, the leader node has also become a performance bottleneck for the entire system. The follower node only receives replication requests and feedback confirmation results during the entire log replication process, with no other network communication. In this paper, we propose an improvement strategy based on the idea of apportionment to reduce the communication complexity of the leader node, improve the algorithm throughput, and optimize the consensus efficiency by using the idle bandwidth resources of the follower node.

2.2.2 Problems with the Leader Election Phase

The Raft algorithm uses a heartbeat mechanism to trigger the leader election. The leader node periodically sends heartbeat packets to the follower node to maintain its leader role. When the follower node does not receive any information about the election timeout for a certain period of time, the leader node is considered to be down in the network and enters the leader election phase.

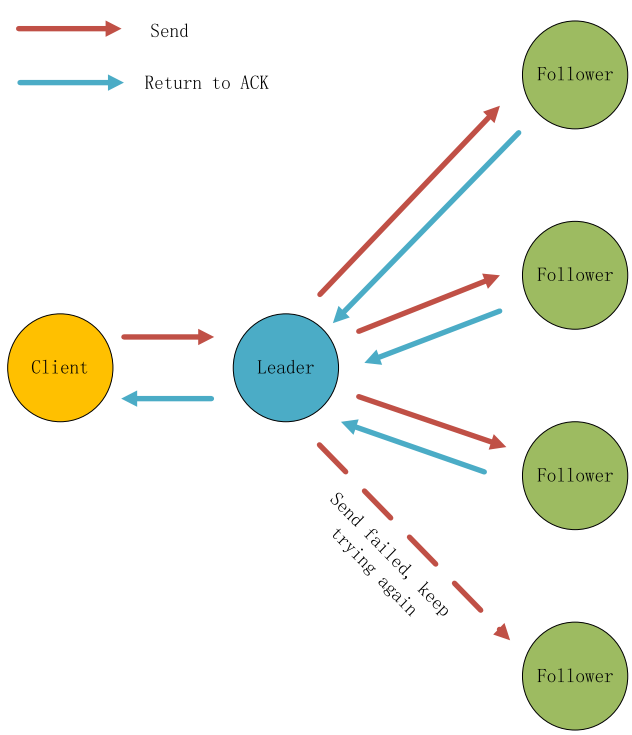

Fig. 1 Diagram of the log replication process

During the leader election, the follower node adds its own term number and switches to candidate state, sends RPC of request vote to surrounding follower nodes to get votes. The candidate node will remain in the current state until one of the following three conditions occurs.

1. Winning the election to become a leader node by obtaining more than half of the votes of the follower node.
2. Other candidate node becomes leader node.
3. In the current term, there are no candidate node that have obtained half of the votes.

Since the Raft algorithm is a consensus algorithm with half of the arbitration at its core, when there are multiple candidate nodes competing for votes, it is easy that each candidate node is unable to obtain more than half of the votes, resulting in multiple rounds of leader election. Although the Raft algorithm uses a random timeout mechanism to restrict multiple candidate nodes from participating in the election at the same time, it is inevitable to have multiple rounds of election conflicts when the network is delayed or the candidate node is down. In this paper, a vote change mechanism is proposed to ameliorate the election time lengthening defect caused by multiple rounds of elections.

2.2.3 Improving the Explanation of Important Concepts in the Algorithm

Since the improved log replication algorithm based on the idea of apportionment changes the replication strategy of Raft algorithm to synchronize only one log at a time, the concepts of epoch and log segment index need to be developed to adapt the new algorithm.

Term The improved algorithm still uses the concept of term that existed in Raft. When one or more candidate nodes compete to become a leader node, the winning node assumes the leader status for the remainder of the term. In exceptional cases, a candidate node that received more than half of the votes cast during the election process may not be able to switch to leader status due to downtime, and the term ends without a leader node being elected. The system will automatically move on to the next term and start a new round of elections. The improved Raft-based algorithm is still designed to ensure that a maximum of one leader is produced through a single term.

Epoch Compared to the Raft algorithm, which is responsible for synchronizing only one log message in a round of consensus, the log replication improvement algorithm based on the apportionment idea introduces the concept of epoch, which can synchronize multiple log messages within a round of consensus by using follower node communication. Thus, epoch means that multiple log messages were successfully committed to the follower node within a round

of synchronization consensus, and is the basic unit of a collection of log messages.

Log segment indexing In the improvement algorithm, each round of data is indexed with log segments. Its purpose is to grasp the capacity size of the log information of the current round, the order of the log set, the follower node owners corresponding to other logs, and other information, so as to facilitate the matching and exchange of the follower node log information set in the log replication phase.

The relationship between term, epoch and segment index is shown in Fig. 2, where term represents the time period where each Leader node is located; epoch represents the relatively fixed log replication blocks for each round of data in the log replication phase; and the log segment index represents the distribution of log resources in each Follower node in a round of data blocks.

3 The Proposed Improved AdRaft

This section elaborates on the design approach to improve the Raft algorithm. Similar to the Raft algorithm, the whole consensus process can be divided into leader election and log replication stages, and the epoch state and segment index state are introduced to ensure the consistency and security of the algorithm.

3.1 Leader Election Improvements Based on Vote Change Mechanism

As Raft's leader election algorithm competes in the same round of election, the votes of follower node can only be cast for one candidate node, and candidate node cannot communicate with each other. Therefore, when there are multiple candidate nodes pulling votes resulting in the number of votes of each candidate node cannot be more than half, the next round of election will have to start again. This paper designs a vote change mechanism, the purpose of which is to introduce vote count state to candidate node, so that within a round of election, candidate node with more votes can capture the votes of candidate node with fewer votes, to prevent the competition among multiple candidate nodes resulting in a deadlock situation.

During the voting election phase, the Raft algorithm is improved to predefine the RequestVote RPC state information describing the initiator and the caller, and the RequestCompare RPC state is added to compare the number of votes between candidate nodes. The four improved state messages are shown in Tables 1, 2, 3 and 4.

The logic that the recipient of the RequestVote RPC needs to implement is as follows.

1. If term < currentTerm, the candidate's term number is less than the follower node's term number, then RequestVote RPC returns (term, false), reminding the candidate that the candidate's term number has expired and rejecting the vote.
2. If term > currentTerm, and the follower node has not previously voted for another candidate node, return (term, true, id, hash), indicating that the votes were delivered to this candidate node during this term. If the follower node has already voted before then, the RequestCompare RPC comparison vote request is triggered.

The specific logic for the recipient implementation of RequestCompare RPC is as follows.

1. If term < currentTerm, that is, the RPC initiator's term number is less than the receiver's, return (term, false), reminding the other party that the term has expired and updating the term number. Otherwise, perform step 2.
2. If the initiator CandidateVoteNum is less than the receiver, the receiver returns (term, false) and the initiator's candidate is automatically changed to the follower state and the subordinate follower node is notified to update the information to which the vote belongs.

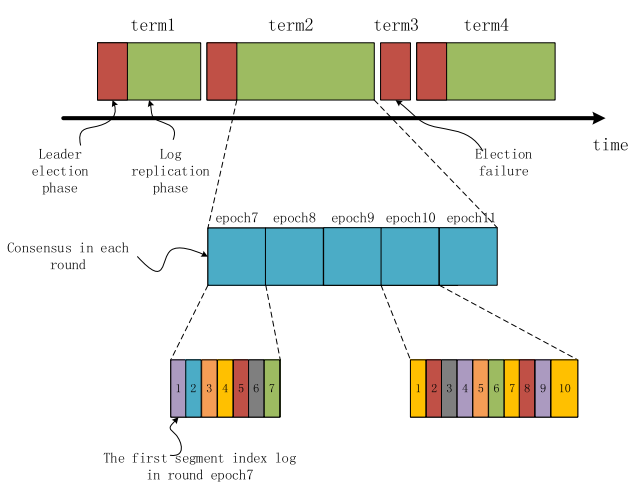

Fig. 2 Schematic diagram of the relationship between term, number of rounds and segment indexing

Table 1 RequestVote RPC'S list of message parameters

Parameter	Description
Term	Current term of the candidate
CandidateId	ID of the candidate requesting a vote
Epoch	The current number of rounds of replication for the candidate
lastLogTerm	Term of the candidate

Table 2 RequestVote RPC'S list of return values

Parameter	Description
Term	Current term number for the candidate to update the local term value
VoteGranted	True if you get the value of the votes, false if you don't
FollowerCandidate	The id of the candidate to follow
LastEpochHash	The hash value of the last round's log set

Table 3 RquestCompare RPC's list of parameters for a request comparison vote message

Parameter	Description
Term	Term number of the candidate
CandidateId	ID of the candidate requesting a vote
CandidateVoteNum	The number of votes held by the candidate requesting the vote
Epoch	The number of rounds in the candidate's current log set
lastLogTerm	Candidate's latest term number

Table 4 RquestCompare RPC'S list of return values

Parameter	Description
Term	Current term number for the candidate to update the local term value
VoteGranted	True if you get the value of the votes, false if you don't

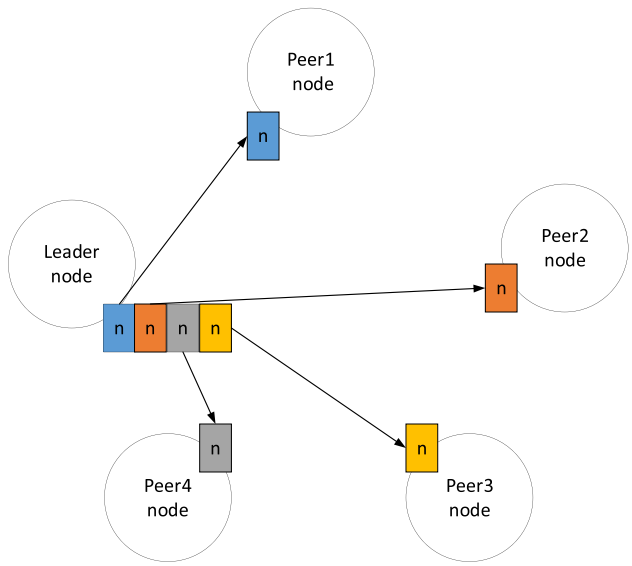

Fig. 3 Leader log replication based on the idea of allocation (1)

3.2 Log Replication Improvements Based on Apportionment Thinking

A log replication improvement strategy is based on the idea of apportionment. In each round of synchronization cycle, the Leader node picks the number of log entries from the buffer less than or equal to the number of locally existing Follower nodes, segment numbers the selected log entries, and initiates a different log replication request to different follower nodes, as shown in Fig. 3.

The follower node receives the log replication message, places the log replication message in the local cache queue, feeds back the acknowledgement result to the leader node, and broadcasts the log replication message sent from the leader node to other follower nodes. In a round of consensus, the leader node will randomly resend log messages to active follower nodes that have not received confirmation messages based on the received follower node confirmation messages, under timing constraints, until it receives confirmation messages from more than half of the nodes in the round, then the log replication process for the round ends. This is shown in Fig. 4.

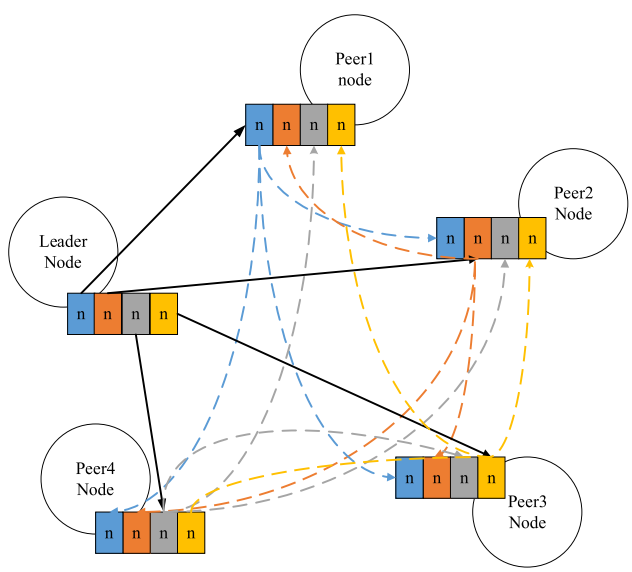

Fig. 4 Leader log replication based on the idea of allocation (2)

Table 5 Persistent state on all nodes

State	Persistent state of all nodes
currentTerm	Node's most recent term number (initialized to 0, continuously incremental)
voterFor	Candidate ID currently on the ballot
Log[]	A collection of log entries: a collection of log entries in rounds, and each round contains the hash value of the previous round's entry set
currentEpoch	Latest number of round of node

Table 6 Frequent state changes on nodes

State	Constant state of all nodes
commitEpoch	Maximum number of log rounds known to have been submitted
lastAppliedEpoch	Number of log entry rounds for the latest local application
lastLogParagIndex[]	The distribution set of logs indexed by each node; log assignment information and node ip for each node in the latest round is recorded

Table 7 Frequent state changes in the leader node

State	Constant state of the Leader node
commitParagIndex[]	Number of nodes identified per log message
AckParagIndex	Number of replication request nodes to complete latest round index logs

As shown in Figs. 3 and 4, the cost of transmitting a message m for the improved algorithm's leader node consensus is only $O\left(\frac{|m|}{N}\right)$ compared to the Raft algorithm, where N is the total number of logs in a round of consensus. In this paper, we will describe the whole algorithm flow and structural framework in more detail. The state of each node is shown in Tables 5, 6 and 7.

In Raft algorithm, the leader node, as a strong leader node, has the unique replication legitimacy of the whole network ledger, that is, all follower nodes must use the log ledger of the leader node as the correct ledger. After improved Raft algorithm by introducing the epoch mechanism, the replication function which can only be initiated by the leader node in one direction is distributed to all follower nodes, so that follower nodes can replicate each other, which undoubtedly increases the complexity of the whole ledger state space. In order to eliminate the complexity problem, this paper divides the entire log ledger into a relatively fixed epoch log collection and a dynamically changing segment index log collection, and then draws on the blockchain concept to generate a hash hash for each fixed round log collection and assigns it to the next round log collection, linking it into a causally related block state. The improved algorithm's leader node collects more than half of the consistent hash values from the previous round to correct the inconsistency of the leader node's log collection and maintain the legitimacy of its own ledger. Leader node observes the RPC confirmation of segment-indexed logs throughout the network through the commitParagIndex parameter, and continuously broadcasts the RPC confirmation of the segment-indexed logs in the segment log collection to the outside. Log messages for which a half consensus has not yet been reached.

Algorithm I, Algorithm II and Algorithm III show the specific steps of the leader node based on the apportionment idea, respectively, which are divided into the broadcast transaction proposal, collect feedback proposal and acknowledge feedback proposal algorithms.

The broadcast transaction proposal algorithm is responsible for traversing the list of nodes, assigning corresponding log messages to different nodes and logging the messages through the mapLog array, as shown in Algorithm I.

Algorithm I Broadcast Transaction Proposal Algorithm for Leader Nodes

```
Input: args[] ( log collection )
Output: log message
Algorithmic pseudocode:
function Replication(args[]){
  var index = 0; // indexing to zero
  var entires = new Entries(); // Creating a new message structure
  var BCL = broadcastList(); // Creating a Broadcast Channel
  for message in args[]{ // Iteration log message collection
    // Initiate a single log replication RPC
    entries.data = message; // Assigning a single log message
    entries.index = index; // Assignment index number
    address = BCL.followerID[index];
    index++;// Index number increment
    mapLog.message[address] = message; // Record sending index and
                                        //value
    // Initiate RPC log replication requests
    BCL.broadcast(term, LeaderId, prevLogTerm, prevLogEpoch, entries,
address);
}}
```

The collection feedback transaction algorithm is responsible for counting the ack acknowledgments returned by the follower node, and randomly selects active nodes to send transaction proposals that do not reach the half-way mark in the timeout. The specific code is shown in Algorithm II.

Algorithm II Leader Node Collection Feedback Trading Algorithm

Input: commit (Feedback confirmation message)
Output: null
Algorithmic Pseudocode:

```
function collectReplicationResponse(followerID, paragIndex, commit){
    if(commit.term > this.cuttentTerm){  // Follower node's term > Leader node's term
        this.currentTerm(commit.term);
        this.toState('follower')  // Changing to follower state
    }else if(commit.LogEpoch != this.currentLogEpoch) //Epoch is different
        Repeat(followerID, mapLog.message[followerID]); // Re-initiate replication RPC          //request
    else if(commit.LogSuccess){ // Feedback received
        commitPatagIndex[paragIndex]++; // Segment index self-addition and feedback            //of results
    }
    // Message timeout mechanism, less than half of the confirmation messages randomly  //selected active nodes re-send
    timeOut(commitPatagIndex);
}
```

The confirmation feedback transaction algorithm is responsible for counting the AckParagIndex information returned from the follower nodes until confirmation feedback is collected from more than half of the nodes. The specific code is shown in Algorithm III.

Algorithm III Leader Node Confirmation Feedback Trading Algorithm

Input: commit (Confirmation Information)

Output: Client feedback argsAck[]
Algorithmic Pseudocode:

```
function ackReplicationResponse(followerID, commit){
    if(this.Majority(AckParagIndex)){ // Determine if most acknowledgements have been //received
        this.LastApplied = epoch; // Update the latest log entries
        BLC.broadcast(lastTerm, lastAppliedEpoch, ack); // Broadcast this round of //information
        return argsAck[];  // Feedback confirmation to the client
    } else if(commit.term > this.cuttentTerm){// Follower node's term > Leader node's //term
        this.currentTerm(commit.term);
        this.toState('follower')  // Changing to follower state
    }else{AckParagIndex++;}  // Confirmation of information for the self-reinforcing //round
}
```

Algorithm IV shows the concrete steps of the Follower node based on the apportionment idea. The core code is divided into AppendLeaderEntries(), a function that receives log copy requests from the Leader node and Follower node, and AllCommit(), a function that returns confirmation information to the Leader node. The specific code is as follows.

Algorithm IV Log replication based on apportionment thinking (Follower node)

Input：Leader/Follower Log Replication Request RPC

Output：Confirmation of feedback
Algorithmic Pseudocode:

```
// Receive log copy request RPC
function AppendLeaderEntires(state, args, cb){
    this.commonState.LeaderId = args.LeaderId;
    if(args.Term < this.currentTerm){  // Check if the leader node tenure is up to date
        callback(false);
    }else{
        if(arg.preLogTerm != this.preLogTerm || arg.preLogEpoch != this.preLogEpoch){// Check if the previous term and number of rounds match the leader //node
            go repair(); // Initiate repair status
        }else{
            this.commonState.pushEntires(args.entries); // Additional log messages
            // Feedback to Leader nodes on deal proposals
            this.ackResponse(this.followerID, args.entries.index, commit);
            if(state == "Leader"){// When the initiated replication request is a Leader //node
                this.sendValue = {args.entries.index, args.entries.data}; // Combo Log //Messages
                BLC.broadcast(term, LeaderId, prevLogTerm, prevLogEpoch, sendValue);// Initiate log RPC requests to other Follower nodes
}
}}}
// Feedback to Leader node to confirm transaction
function AllCommit(){
    if(this.allEpoch()){ // Check that the current round of log messages is complete
        this.ackReponse(this.followerID, commit); // Back to confirmation information
}}
```

In Fig. 5, the ledger consists of sequentially numbered blocks of log rounds. Each node's log entry contains four attributes: round number(epoch), term number, ParagIndex number and log information. Epoch represents the smallest integer unit of the submitted log set and varies according to the number of available follower nodes in the network. Term number is the number created by the leader at the time of inauguration and corresponds to the topmost sequence of numbers in the log entry. ParagIndex number is present in each log set, but the non-fixed ParagIndex number is the set of log index numbers that the leader node will agree on within the latest epoch. If the set of log messages for the latest epoch can be executed in rounds, it is considered ready for commit.

The leader node has the power to determine when the latest epoch of log entry collections are committed. Once the AckParagIndex parameter of the current round of log replication request RPC reaches half or more, then the current round of log entry collection can be committed. For example, in Fig. 5, the set of log entries of epoch4 has records in

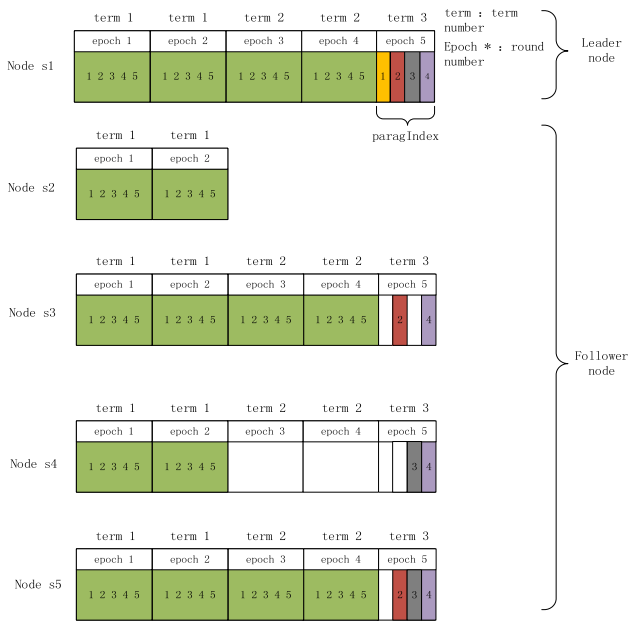

Fig. 5 Schematic diagram of algorithmically improved log replication based on apportionment idea

three of the nodes, and epoch4 is considered to be committed. Epoch5 cannot be committed because S2 is down and each node does not agree on a complete set of log entries.

The leader node based on apportionment idea will use the consistent hash value of the collection of logs of more than half of the nodes in the previous round as the basis of correctness of its own node when it completes the election. If the leader node's ledger is found to be faulty, it can correct its own log set by continuously interacting with other follower nodes and backtracking more than half of the hash values to ensure the strong consistency of the leader node. If the follower node's ledger is inconsistent but not down, it can request the replication of the ledger to the leader node that has completed the log set backtracking without affecting the follower node's normal participation in the latest epoch of log replication requests. For example, in Fig. 5, S4 can still participate in the log consensus of the round epoch5 although the log information of the epoch3 round and the epoch4 round is vacant. Log replication RPC is requested at the Leader node for the log information of the vacant rounds.

The implementation details of the log replication request AppendEntries RPC at the leader node of this paper are shown in Tables 8, 9 and 10. The caller of the AppendEntries RPC is the Leader node and the receiver is the follower node. The AppendEntries RPC also inherits the function of the Raft algorithm and acts as a heartbeat packet for the leader node in addition to making log copy requests.

Improved Raft due to the fact that the follower node can pass information properties and also has its own AppendFollow RPC request, and the follower node is designed to only externally push the copy information sent by the Leader node without the return value type. The details of the implementation are shown in Table 8.

Table 8 AppendEntries RPC replication request for leader node

Parameter	Description
Term	Term number
LeaderId	The ID of the leader
prevLogTerm	The term number of the previous log message collection
prevLogEpoch	The round number of the previous log message set
entries[]	One of the set of messages in the current round, empty when sending heartbeat packets
FollowerId	Follower's ID

4 Consistency Arguments for AdRaft Algorithm

The first few vignettes describe how the improved Raft algorithm performs Leader election and log replication. However, the current encounter does not guarantee that every log entry for every node is replicated and executed in the same order. For example, a follower may enter an unavailable state due to downtime when the leader has already submitted several requests for log replication, causing inconsistencies between ledgers. This section complements and refines the rules for Improved Raft by describing the various limiting mechanisms and providing a brief proof.

4.1 Mechanisms Design of the Algorithm

Algorithm Mechanism 1: improvements to the Raft algorithm must collect the results of more than half of the nodes for the current round of logging consensus before each round of logging collection is submitted. For every round of log consensus that has been committed in the network, there are at least half of the nodes that have local commit results for that round of consensus.

Algorithm mechanism 2: each round of the improved Raft algorithm's commit log set will necessarily contain the hash value of the previous round's log set. The current round log collection has a chained data structure and causality with the previous round log collection.

4.2 Completeness of the Leader Node

This paper follows the idea that the leader of Raft's algorithm must have a uniquely correct ledger, but the leader node of the improved algorithm is not a dictatorial leader node. Leader node's ledger correctness comes from the

Table 9 AppendEntries RPC return value for the leader node

Parameter	Description
currentTerm	Current tenure number for comparing the size of the tenure number between the Leader leader and Follower nodes
LogSuccess	Acknowledge receipt of one of the log messages in the log collection
LogParagSuccess	Acknowledgement of receipt of all log collection messages for this round

Table 10 AppendFollow RPC replicate request

Parameter	Description
Term	Term number
LeaderId	Leader's ID
prevLogTerm	Term numbers for previous rounds of log message collections
prevLogEpoch	The round number of the previous round of log message collections
sendValue	Send log messages received from the Leader node

self-correction of the ledger after the election is complete. The leader node maintains the correctness of its own last round of log sets by collecting the hash values of more than half of the consensus of the previous round of log sets by heartbeat packets. According to algorithm mechanism 1, each round of correct log collection exists in more than half of the log nodes, so the correctness of the previous round of log collection of Leader node is judged by the hash of more than half of the collection. According to the algorithm mechanism 2, each round of log collection exists in a chained data structure, that is, it can correct the inconsistency of the previous round of log collection through the retrospective function, until the local log collection of the Leader node in a certain round matches the consistency of the log collection of more than half of the network nodes. Leader node can then be broadcasted across the network, and become a complete leader node.

4.3 Electoral Restrictions

The improved Raft algorithm organizes a candidate to win an election by using the term number and the round number. Unlike the Raft algorithm, the Leader node does not enforce the integrity of forward log entries. The improved Raft algorithm instead rejects candidates whose terms have expired by term number. The RPC for requesting a vote implements the restriction that the RPC contains information about the candidate's tenure, and then the following node rejects vote requests that have a lower tenure number than its own.

Improved Raft finds the log set that matches the latest round of consensus reaching by comparing the round numbers. The latest round of log set consensus reaching is guaranteed by more than half of the node acknowledgement information in Algorithm Mechanism 1.

4.4 Overlay Issues for Log Collections

During the log replication phase there is a situation where the Leader node crashes for some reason when it receives more than half of the replicated log acknowledgement requests, before it is sure it can commit. Alternatively, the Leader node crashes while the latest round of replication requests is in progress. In this paper, two concepts are used to ensure the consistency of the ledger, namely the relatively fixed number of rounds term of the log set and the changing segment index value of the log set. Improved Raft uses a fallback mechanism to guarantee the inconsistency of the segment index ledger due to Leader node crashes. After completing the election, each node abandons the recovery of the segment-indexed log set for the latest round and reverts back to the previously fixed round number log book.

5 Performance Testing

The system build and test environment are mainly implemented through Linux virtual machine services. The specific software and hardware version parameters and configuration information are shown in Table 11. This section uses Hyperledger's official testing tool Caliper to test and verify both log replication and leader election by setting up the baseline configuration, network profile and test data, respectively. Meanwhile, throughput and latency are important indicators to measure the performance of the blockchain network, and the effectiveness and optimization of this scheme are demonstrated by comparing it with the original Raft algorithm.

Table 11 Environmental parameters for software and hardware

Software and hardware environment	Model
Server (PC)	Ubuntu-16.04, inter core i5–8500 processor, Memory 24 g, hard disk capacity 1T
Hyperledger fabric	1.4.3
Docker	17.03.2-ce
Performance testing tools	Caliper

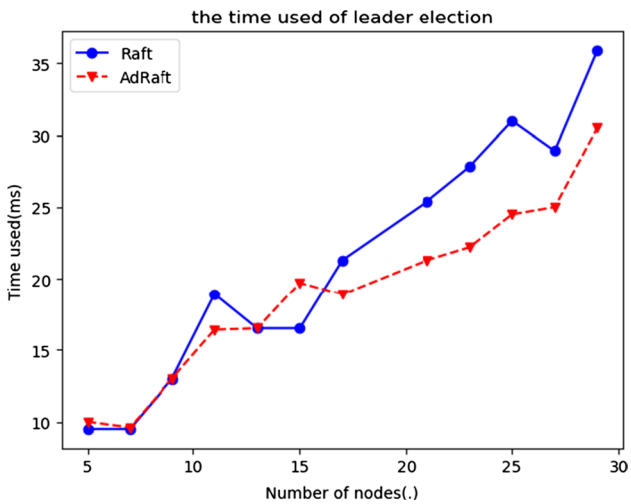

Fig. 6 Time spent on elections at different nodes

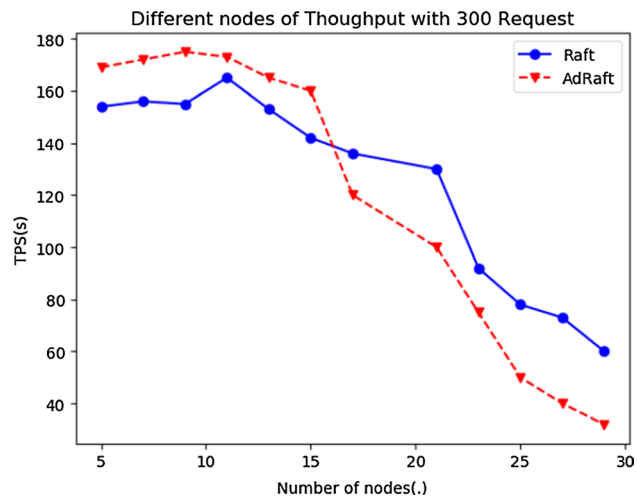

Fig. 7 Network throughput with different number of nodes

5.1 Performance Testing During the Leader Election Phase

The Leader election algorithm based on the vote change mechanism, compared to the original algorithm, proves that except for the multiple rounds of elections caused by the Candidate node crashing when getting more than half of the votes, the AdRaft algorithm always achieves consensus in one round of elections so that the Candidate node with the highest number of votes becomes the Leader node. Figure 6 shows the time consumed by the AdRaft and Raft algorithms for Leader elections with different numbers of nodes.

The Leader election based on vote change mechanism and the original algorithm shows a linear increase in the election time with the increase in the number of nodes, but the Raft algorithm has multiple rounds of election in 11 nodes and 25 nodes, which leads to abrupt increase in the election time. The overall election time of the Leader election algorithm based on the vote change mechanism is less than the original algorithm.

5.2 Performance Testing of the Log Replication Phase

AdRaft uses an improved log replication algorithm based on the idea of apportionment in the log replication phase to utilize the idle communication resources of the peer nodes, and the leader node sends different log messages to other peer nodes via the latest round (epoch).

Figure 7 depicts the throughput performance exhibited by AdRaft algorithm and the original Raft algorithm to reach consensus in 300 transaction requests for different numbers of nodes. In a Fabric blockchain consisting of 5 to 13 nodes, both the AdRaft and Raft algorithms fluctuate in a stable throughput range, and it is known from Fig. 7 that the throughput of the AdRaft algorithm is higher than that of the Raft algorithm. However, when the network size is larger than 15 nodes, the throughput of AdRaft algorithm is lower than Raft algorithm in the same node size, the reason for this performance difference is that the AdRaft algorithm uses a log replication strategy with apportionment ideology, which causes the Leader node and the peer node to incur significant performance overhead simultaneously during the algorithm run resulting in severe throughput degradation compared to the Raft algorithm.

Figure 8 depicts the comparative latency results of the AdRaft and Raft algorithms for 300 transaction requests for Fabric networks with varying numbers of nodes. The delay of AdRaft decreases from 23.65 to 15.96 s for networks with less than 15 nodes, while the delay of Raft fluctuates in the range of [20, 23] seconds, and increases linearly for networks with more than 15 nodes due to the increasing performance overhead of docker.

Figure 9 depicts the comparison of the AdRaft and Raft algorithms' throughput with the number of transaction requests in a seven-node network. The comparison graph shows that the throughput performance of the AdRaft and Raft algorithms increases with increasing transaction requests at less than 250 transaction requests. the Raft algorithm reaches a peak of 159.03 TPS at 250 requested transactions, while the AdRaft algorithm reaches a peak at 300 transaction requests due to the larger communication characteristics shared by the larger number of nodes 175.6 TPS. After which the throughput of both algorithms gradually leveled off. The AdRaft algorithm improves 5.8% over original Raft in terms of throughput performance.

Figure 10 shows a comparison of the latency of the AdRaft algorithm and the Raft algorithm with the number of transaction requests in a seven-node Fabric network. The delay of the AdRaft algorithm is higher than that of the Raft algorithm for less than 64 Request requests. The reason for this is that the apportionment strategy algorithm boosts the latency time due to the additional validation overhead to ensure book consistency. Whereas for more than 100 Request requests, the $O\left(\frac{|m|}{N}\right)$ communication optimization provided by the AdRaft algorithm makes the latency lower than the Raft algorithm. In terms of latency, AdRaft is 1.3% shorter than the original algorithm.

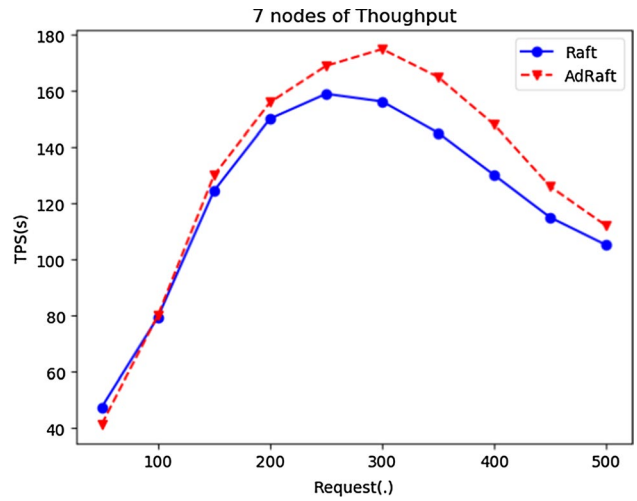

Fig. 9 Concurrent transaction throughput under seven nodes

6 Conclusions

This paper proposes an improvement for Raft algorithm based on Hyperledger Fabric framework. The improved Raft algorithm designs a vote change mechanism in the leader election phase, and proposes a leader election algorithm based on the vote change mechanism to reach a consensus in a round of election; it adopts the communication mechanism of apportionment idea in the log replication phase, and makes full use of the idle channel resources of follower nodes to synchronize multiple log messages in a round to improve the throughput of leader nodes, and consensus efficiency. This paper also demonstrates the consistency of the adraft algorithm and proves the usability and effectiveness

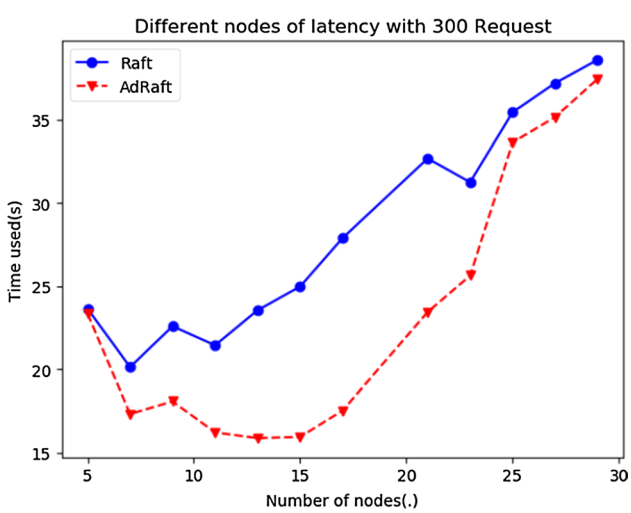

Fig. 8 Network delay under different number of nodes

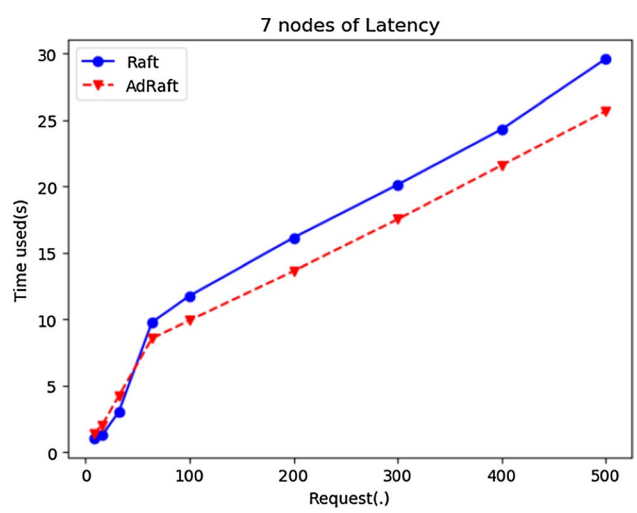

Fig. 10 Concurrent transaction latency with seven nodes

Table 12 Comparison of improved raft and raft

Contrast Points	Raft	Improved Raft
Leader type	Strong leader type, can only launch the replication function from the leader node in one direction	The leader's rights are delegated, and follower nodes can replicate each other
Leader election	Deadlock caused by multiple candidate competition during leader election	Vote change mechanism is introduced during election voting to prevent multiple candidate competition
Election restriction	The election leader is elected according to the size of the term value, strictly greater than the former leader's term	Leaders are elected by using a combination of term number and epoch number screening
Safety	The leader must eventually store all committed log entries, and the elected leader never overwrites entries that already exist in the local log	Adopt a fallback mechanism to ensure that segment index ledger inconsistencies are caused by leader node crashes
Availability	Relies on timing mechanism	Relies on timing mechanism
Performance	Leader election time	The overall time spent is greater than the election time for the improved raft	Overall time is less than Raft
	Throughput and latency of the log replication phase	Not as good in terms of throughput and latency as the improved raft	In terms of throughput, it is 5.8% better than Raft, and in terms of latency, it is 1.3% shorter than Raft

of the algorithm. Adraft algorithm is optimized from these two aspects to ensure that the Hyperledger Fabric blockchain network has better performance than the original Raft algorithm when it is applied.

In general, both the improved raft and the original raft need some restriction mechanisms to complement and improve them. Table 12 shows the comparative analysis of the original raft and the improved raft.

Finally, this paper proves the effectiveness of the solution by comparing and verifying with the Adraft algorithm in terms of log replication and leader election, respectively.

References

1. Lamport, L.; Shostak, R.; Pease, M.: The Byzantine generals problem. ACM Trans. Program. Lang. Syst. **4**(3), 382–401 (1982)
2. Castro, M.; Liskov, B.: Practical Byzantine fault tolerance and proactive recovery. ACM Trans. Comput. Syst. **20**(4), 398–461 (2002)
3. Narayanan, A.; Bonneau, J.; Felten, E., et al.: Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction, p. 27–30. Princeton University Press, Princeton, NJ (2016)
4. Sheikh, H.; Meer, A.R.; Rizwan, F.: Smart contract development, adoption and challenges: the powered blockchain. Int Res J Adv Eng Sci **4**(2), 321–324 (2019)
5. The ZILLIQA Technical Whitepaper[EB/OL]. https://docs.zilliqa.com/whitepaper.pdf
6. Gilad, Y.; Hemo, R.; Micali, S., et al.: Algorand: scaling byzantine agreements for cryptocurrencies. In: 2017 ACM SIGOPS 26th Symposium on Operating Systems Principles, pp. 51–68. ACM, Shanghai (2017)
7. Li, C.; Li, P.; Zhou, D., et al.: Scaling nakamoto consensus to thousands of transactions per second. Comput. Sci. (2018)
8. Wang, J.; Wang, H.: Monoxide: scale out blockchains with asynchronous consensus zones. In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI), pp. 95–112. Boston, MA, United States (2019)
9. James, A.; Myers, D.S.; Barbara, L., et al. HQ replication: a hybrid quorum protocol for byzantine fault tolerance. In: SOSP' 07: Proceedings of the 21st ACM symposium on Operating systems principles. New York, USA. 2006: 177–190.
10. Ramakrishna, K.; Lorenzo, A.; Mike, D., et al.: Zyzzyva: speculative byzantine fault tolerance. ACM Trans. Comput. Syst. **27**(4), 45–58 (2009)
11. Liu, J.; Li, W.; Karame, G., et al.: Scalable byzantine consensus via hardware-assisted secret sharing. IEEE Trans Comput **68**(1), 139–151 (2018)
12. Baird, L.: The Swirlds hashgraph consensus algorithm: fair fast byzantine fault tolerance[EB/OL]. http://www.swirlds.com/downloads/SWIRLDS-TR-2016-01.pdf
13. Andrew M., Xia Yu, Kyle C., et al. The honey badger of BFT protocols. In: CCS '16: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 31–42. ACM, Vienna (2016)
14. Lamport, L.: The part-time parliament. ACM Trans. Comput. Syst. **16**(2), 133–169 (1998)
15. Junqueira, F.P.; Reed, B.C.; Serafini, M.: Zab: high-performance broadcast for primary-backup systems. In: IEEE/IFIP 41st International Conference on Dependable Systems and Networks (DSN), pp. 245–256. IEEE Press, Hong Kong, China (2011)

16. Burrows, M.: The Chubby lock service for loosely-coupled distributed systems. In: 7th Symposium on Operating Systems Design and Implementation (OSDI '06), pp. 335–350. USENIX Association, Seattle (2006).
17. Ongaro, D; Ousterhout, J.: In search of an understandable consensus algorithm. In: USENIX Annual Technical Conference. [s.l.], pp. 305–319. USENIX (2014)
18. Wang, Z.; Li, T.; et al.: CRaft: an erasure-coding-supported version of raft for reducing storage cost and network cost. In: 18th USENIX Conference on File and Storage Technologies (FAST '20), pp. 303–344. USENIX, Santa Clara (2020)
19. Tan, D.; Hu, J.; Wang, J.: VBBFT-Raft: an understandable blockchain consensus protocol with high performance. In: 2019 IEEE 7th International Conference on Computer Science and Network Technology (ICCSNT), pp. 111–115. IEEE, Dalian (2019)
20. Wang, R.; Zhang, L.; Zhou, H.; Xu, Q.: A byzantine fault tolerance raft algorithm combines with BLS signature. J. Appl. Sci. **38**(01), 93–104 (2020)
21. Wang, R.; Zhang, L.; Xu, Q., et al. K-bucket based raft-like consensus algorithm for permissioned blockchain. In: 2019 IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS), pp. 996–999. IEEE, Tianjin (2019)
22. Dai, K.: Design of Optimized Log Replication Distributed Consensus Algorithm OLR-Raft Based on Raft. Huazhong University of Science and Technology, Wuhan (2017)
23. Zhike, W.: Research on Consistency Problem of Distributed System Based on Raft Consensus Algorithm. Xidian University, Xi'an (2019)